JSON - Complete Developer Notes

1. WHAT EXACTLY IS JSON?

JSON (JavaScript Object Notation) is a lightweight text format used for storing and exchanging structured data.

JSON is language-independent.

Although it originated from JavaScript syntax, today every major language supports JSON.

Examples:

```
{
    "name": "Chandan",
    "age": 27,
    "isEmployee": true
}
```

The same JSON can be read by:

- Java
- C++
- Python
- Go
- JavaScript
- Rust
- C#

This interoperability is one of the biggest reasons for its success.

Why Do We Need JSON?

Imagine:

Frontend:

```
{
    name: "Chandan"
}
```

Backend:

```
User{name="Chandan"}
```

Database:

```
name=Chandan
```

All systems store data differently.

We need a common format.

JSON became that common language.

Before JSON: XML Dominated Everything

Around the late 1990s and early 2000s, XML was the standard.

Example:

```
<person>
    <name>Chandan</name>
    <age>27</age>
</person>
```

Problems:

- Too verbose
- Large payload size
- Harder to read
- More parsing overhead

Equivalent JSON:

```
{
    "name":"Chandan",
    "age":27
}
```

Much smaller.

Much cleaner.

Developers loved it.

Why Did JSON Explode?

Three reasons.

1. HUMAN READABLE

Anyone can understand:

```
{
    "city":"Jaipur"
}
```

within seconds.

---

2. JAVASCRIPT SUPPORT

JavaScript could directly convert objects to JSON.

```
JSON.stringify(user)
```

and back:

```
JSON.parse(data)
```

Since browsers run JavaScript, JSON became the natural choice.

---

3. REST APIS BECAME POPULAR

When REST APIs exploded:

```
GET /users/1
```

Response:

```
{
    "id":1,
    "name":"Chandan"
}
```

JSON became the default API language.

JSON Structure

JSON only has two containers.

OBJECT

Key-value pairs.

```
{
    "name":"Chandan"
```

```
}
```

ARRAY

Ordered collection.

```
[
    "Java",
    "C++",
    "Python"
]
```

Everything in JSON is built using these two.

JSON Data Types

JSON supports:

STRING

```
"name":"Chandan"
```

NUMBER

```
"age":27
```

BOOLEAN

```
"isEmployee":true
```

NULL

```
"manager":null
```

OBJECT

```
{
    "city":"Jaipur"
}
```

ARRAY

```
[
    "A",
    "B"
]
```

Notice:

No date type.

No map type.

No set type.

No function type.

JSON Rules Interviewers Expect

Keys must be strings.

Valid:

```
{
    "age":27
}
```

Invalid:

```
{
    age:27
}
```

Double quotes only.

Valid:

```
"name":"Chandan"
```

Invalid:

```
'name':'Chandan'
```

No trailing commas.

Invalid:

```
{
    "name":"Chandan",
}
```

Internal Reality: JSON Is Just Text

Most developers stop here.

Big-tech interviews don't.

When you see:

```
{
    "name":"Chandan",
    "age":27
}
```

you think:

"Object"

Computer thinks:

```
{
"
n
a
m
e
"
:
...
```

JSON is merely text.

Nothing more.

What Actually Travels Over the Network?

Suppose frontend sends:

```
{
    "name":"Chandan",
    "age":27
}
```

Network does NOT send an object.

Network sends bytes.

Example:

```
7B
22
6E
61
6D
65
...
```

These are UTF-8 encoded bytes.

This distinction becomes important in distributed systems.

Serialization

Object → JSON Text

Example:

```
User user
```

becomes

```
{
    "name":"Chandan"
}
```

This process is called:

SERIALIZATION

Deserialization

JSON Text → Object

```
{
    "name":"Chandan"
}
```

↓

```
User user
```

called:

DESERIALIZATION

Every API call performs both.

Why Big Tech Cares About Serialization

Imagine:

1 Billion API requests/day.

If serialization costs:

```
0.5 ms/request
```

Total cost becomes enormous.

Even tiny inefficiencies become expensive.

Problem #1: Repeated Field Names

JSON:

```
{
    "name":"A"
}
```

```
{
    "name":"B"
}
```

```
{
    "name":"C"
}
```

The string:

```
"name"
```

keeps getting transmitted.

Again.

Again.

Again.

At scale, this wastes bandwidth.

Problem #2: Parsing Cost

Parser must examine:

```
{
}
:
,
"
```

continuously.

This is CPU work.

Google serving billions of requests cannot ignore CPU cost.

Problem #3: Larger Payloads

JSON contains metadata.

Example:

```
{
    "name":"Chandan",
    "age":27
}
```

contains:

```
{
}
:
,
"
```

which are not actual business data.

They are formatting characters.

Binary protocols avoid much of this overhead.

Problem #4: Memory Allocation

During parsing:

```
{
    "name":"Chandan"
```

```
}
```

parser creates:

- strings
- objects
- arrays
- buffers

More allocations = more GC pressure.

Important in Java and C# systems.

Problem #5: Schema Validation

JSON itself does not define structure.

You may receive:

```
{
    "age":"twenty"
}
```

instead of:

```
{
    "age":20
}
```

Need validation layers.

Why Protobuf Was Created

Google faced massive scale.

JSON became expensive.

Google created:

Protocol Buffers

Instead of:

```
{
    "name":"Chandan",
    "age":27
}
```

fields are encoded as compact binary identifiers.

Benefits:

- Smaller payload
- Faster parsing
- Lower CPU
- Lower bandwidth

JSON vs Protobuf

Feature	JSON	Protobuf
Human Readable	✅	❌
Easy Debugging	✅	❌
Bandwidth	❌	✅
Parsing Speed	❌	✅
Browser Friendly	✅	❌
API Friendly	✅	⚠️
Microservices	Good	Excellent

Where JSON Is Still King

Despite all its flaws:

JSON remains dominant because:

- Easy to read
- Easy to debug
- Easy to log
- Supported everywhere
- Great developer experience

Therefore:

External APIs → JSON

Internal high-performance services → Often Protobuf/Avro\

Big Tech Questions

Q1. Difference between Serialization and Deserialization?

SERIALIZATION

Object → JSON/Text/Binary

Example:

```
User user("Chandan", 27);
```

becomes

```
{
  "name":"Chandan",
  "age":27
}
```

Purpose:

- Send data over network
- Store data in files
- Save data in cache

DESERIALIZATION

JSON/Text/Binary → Object

```
{
  "name":"Chandan",
  "age":27
}
```

becomes

```
User user;
```

INTERVIEW ONE-LINER

> Serialization converts an in-memory object into a transferable format, while deserialization reconstructs the object from that format.

Q2. Why is JSON text-based?

Because humans can easily read and debug text.

Compare:

JSON:

```
{
  "name":"Chandan"
}
```

vs Binary:

```
011010010101011001...
```

Advantages:

✅ Human readable

✅ Easy debugging

✅ Easy logging

✅ Language independent

Tradeoff:

❌ Larger payload

❌ Slower parsing

INTERVIEW ONE-LINER

> JSON is text-based because it prioritizes readability, interoperability, and ease of debugging over raw performance.

Q3. Why is Protobuf faster than JSON?

JSON parser must process:

```
{
}
:
,
"
```

every time.

Protobuf uses compact binary encoding.

Instead of:

```
{
  "name":"Chandan",
  "age":27
}
```

it stores:

```
field1=value
field2=value
```

in binary form.

Benefits:

- Less data to read
- Less parsing work
- Less memory allocation
- Better cache efficiency

INTERVIEW ONE-LINER

> Protobuf is faster because it uses compact binary encoding and avoids expensive text parsing.

Q4. Why does JSON consume more bandwidth?

Because field names are transmitted repeatedly.

Example:

```
{
  "name":"A"
}
```

```
{
  "name":"B"
}
```

```
{
  "name":"C"
}
```

The string:

```
"name"
```

is sent every time.

Plus extra characters:

```
{
}
:
,
"
```

also travel over network.

INTERVIEW ONE-LINER

> JSON consumes more bandwidth because it repeatedly transmits field names and formatting characters as text.

Q5. When would you choose JSON over Protobuf?

Choose JSON when:

PUBLIC APIS

Example:

```
GET /users
```

Developers can inspect responses easily.

BROWSER COMMUNICATION

JavaScript handles JSON naturally.

LOGGING

Logs remain human-readable.

DEBUGGING

Easy to inspect in Postman.

INTERVIEW ONE-LINER

> I choose JSON when developer experience, readability, and interoperability are more important than performance.

Q6. Why does Google use Protobuf internally?

Google handles:

- Billions of requests
- Huge datasets
- Thousands of services

Even saving:

```
1 ms
```

per request becomes massive.

Goals:

✅ Smaller payloads

✅ Lower network cost

✅ Faster serialization

✅ Faster deserialization

✅ Strong schema support

INTERVIEW ONE-LINER

> Google uses Protobuf internally because at massive scale, reducing payload size and CPU overhead saves significant infrastructure cost.

Q7. What is the role of UTF-8 in JSON?

Network does not send characters.

Network sends bytes.

JSON text:

```
{
  "name":"Chandan"
}
```

must be converted into bytes.

UTF-8 defines how characters become bytes.

Example:

```
A
```

↓

```
41
```

(hex)

Modern JSON systems generally use UTF-8.

INTERVIEW ONE-LINER

> UTF-8 is the character encoding that converts JSON text into bytes for storage and network transmission.

Q8. Can JSON represent binary data directly?

No.

JSON supports:

- String
- Number
- Boolean
- Null
- Object
- Array

Binary data is not a JSON type.

Common solution:

Convert binary → Base64 string

Example:

```
{
  "image":"iVBORw0KGgoAAA..."
```

```
    }
```

Receiver:

Base64 → Binary

INTERVIEW ONE-LINER

> JSON cannot directly represent binary data, so binary content is typically encoded as a Base64 string.